

Synchronous Mapping and Navigation

Synchronous Mapping and Navigation

1. Course Content
2. Scenario Introduction
3. Start the Agent
4. Run the Example
 - 4.1 Start the Mapping and Navigation Node
 - 4.2 Save the Map
5. Source Code Analysis
 - 5.1 View TF Tree
 - 5.2 Launch File

1. Course Content

1. Run the sample program to perform simultaneous mapping and navigation in an unknown environment without a pre-existing grid map.

2. Scenario Introduction

- Suitable for scenarios where long-term map storage is not required and the task only needs to be completed once. (After the task is completed, the map can be saved for future use with `map_saver_cli`).
- Suitable for when a robot first enters an unmodeled scene, autonomously building an environmental map by being given a target pose and completing a specified area traversal or reaching a target point.

3. Start the Agent

Note: If it is already running, you do not need to start it again.

Enter the command in the vehicle's terminal:

```
start_agent
```

The terminal will print the following information, indicating a successful connection:

```
mircoROS_Agent
subscriber created | client_key: 0x10D9EFBC, subscriber_id: 0x000(4), partici
pant_id: 0x000(1)
[1765183764.822476] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x10D9EFBC, datareader_id: 0x000(6), subscri
ber_id: 0x000(4)
[1765183764.826313] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x10D9EFBC, topic_id: 0x006(2), participant_
id: 0x000(1)
[1765183764.829482] info | ProxyClient.cpp | create_subscriber | s
subscriber created | client_key: 0x10D9EFBC, subscriber_id: 0x001(4), partici
pant_id: 0x000(1)
[1765183764.834154] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x10D9EFBC, datareader_id: 0x001(6), subscri
ber_id: 0x001(4)
[1765183764.838233] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x10D9EFBC, topic_id: 0x007(2), participant_
id: 0x000(1)
[1765183764.842504] info | ProxyClient.cpp | create_subscriber | s
subscriber created | client_key: 0x10D9EFBC, subscriber_id: 0x002(4), partici
pant_id: 0x000(1)
[1765183764.847468] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x10D9EFBC, datareader_id: 0x002(6), subscri
ber_id: 0x002(4)
```

4. Run the Example

4.1 Start the Mapping and Navigation Node

For Raspberry Pi 5 users, if the ROS container is not started, you need to start it first,

```
bringup_m3
```

Open a terminal inside the container,

```
exec_m3
```

(ORIN board)

```
ros2 launch m3_bringup nav2.launch.py slam:=True
```

(RDK & PI5 boards) Run in separate steps, with visualization launched on the accompanying virtual machine `Rosmaster-M3-Ubuntu22.04`,

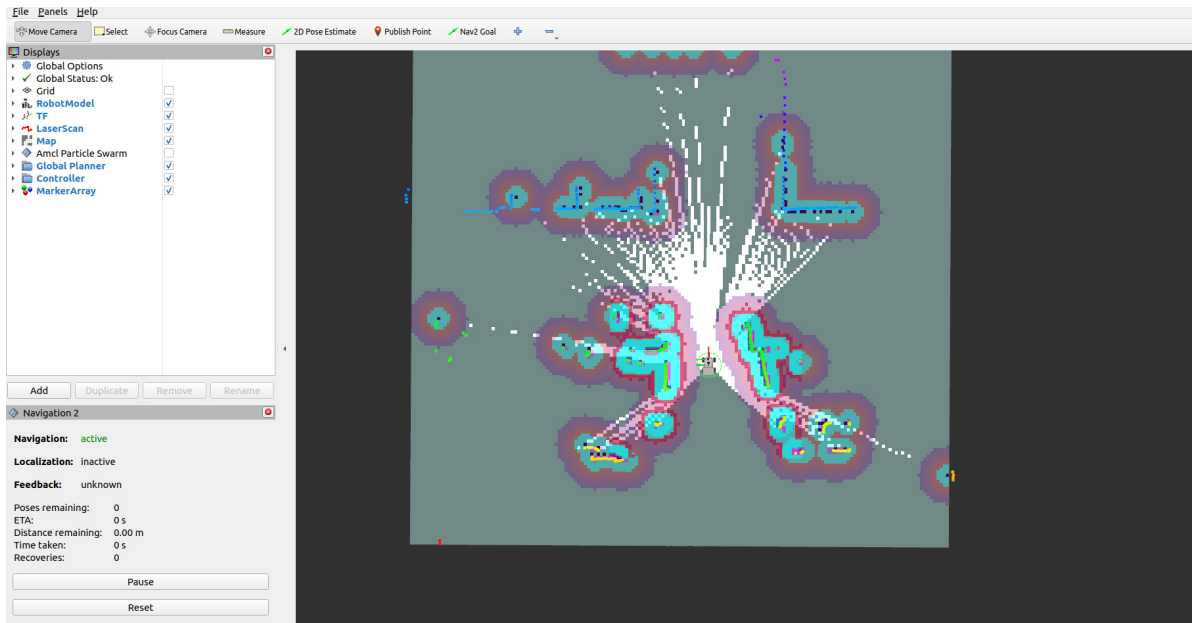
```
# On the virtual machine
rviz2 -d /home/yahboom/yahboomM3_ws/user_ws/src/m3_bringup/rviz/nav2.rviz
# On the host machine
ros2 launch m3_bringup nav2.launch.py slam:=True gui:=False
```

[!TIP]

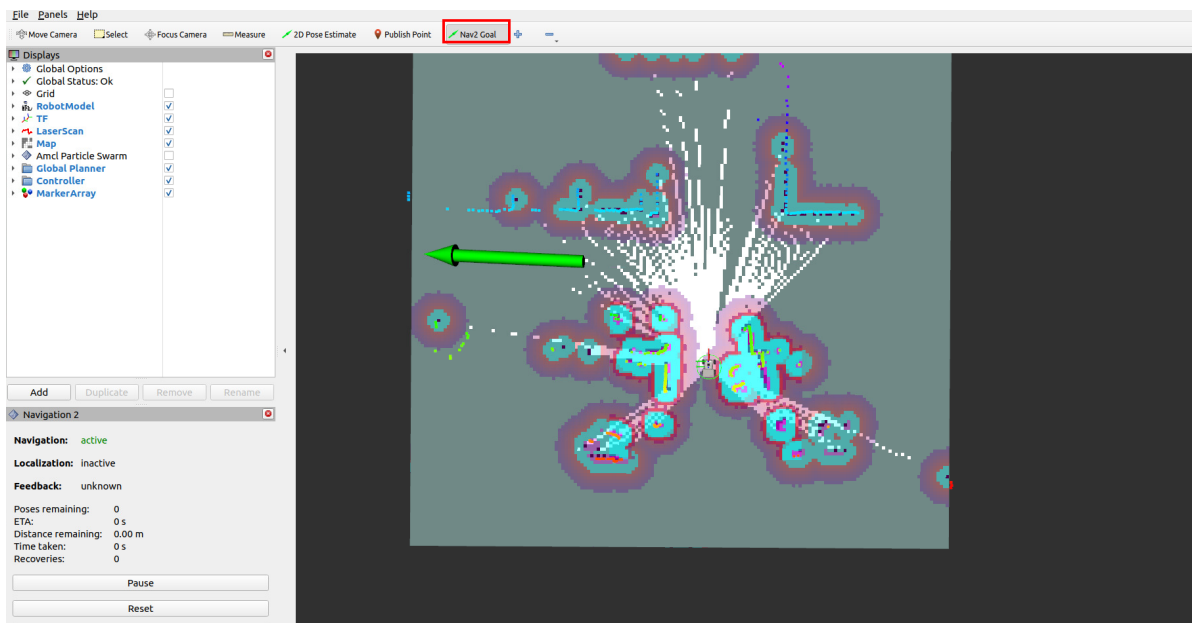
Launch Parameter Descriptions

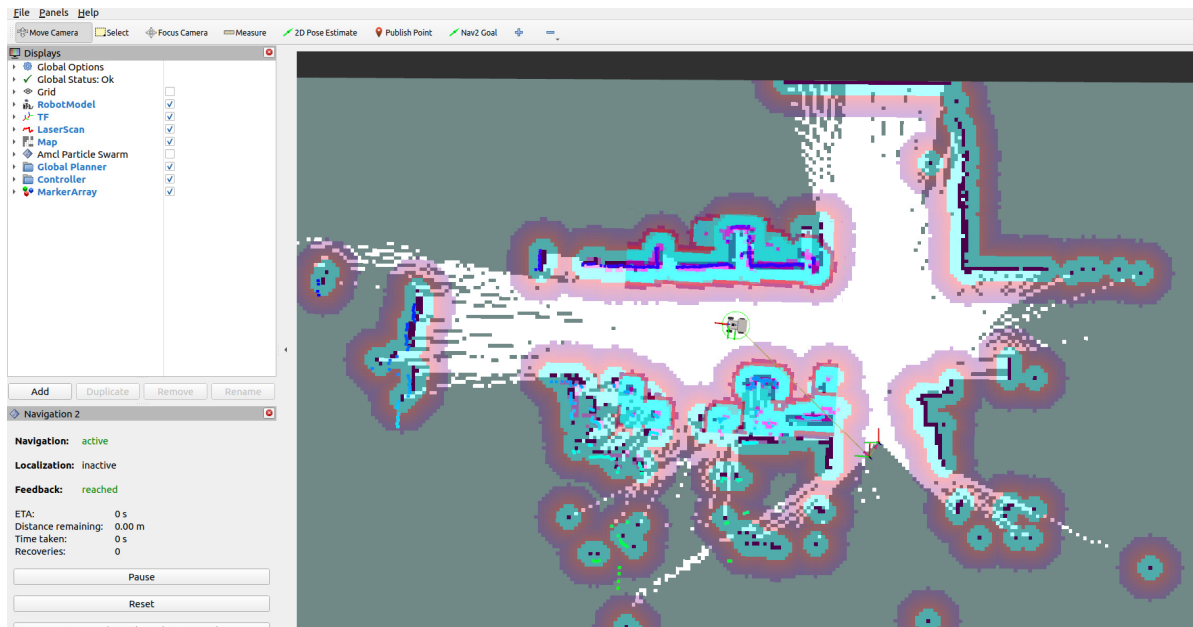
- `slam`

- Whether to start the simultaneous mapping and navigation mode, default is `False`.



- Use the `Nav2 Goal` tool to give a target pose, and the robot can perform navigation while building the map.





4.2 Save the Map

- The commands for saving the grid map and the pose map are the same as in the previous lessons and will not be repeated here.
- Save the grid map:

```
ros2 launch m3_bringup save_map.launch.py
```

- Save the pose graph map in `posegraph` format:

```
ros2 launch m3_bringup save_map.launch.py map_type:=posegraph
```

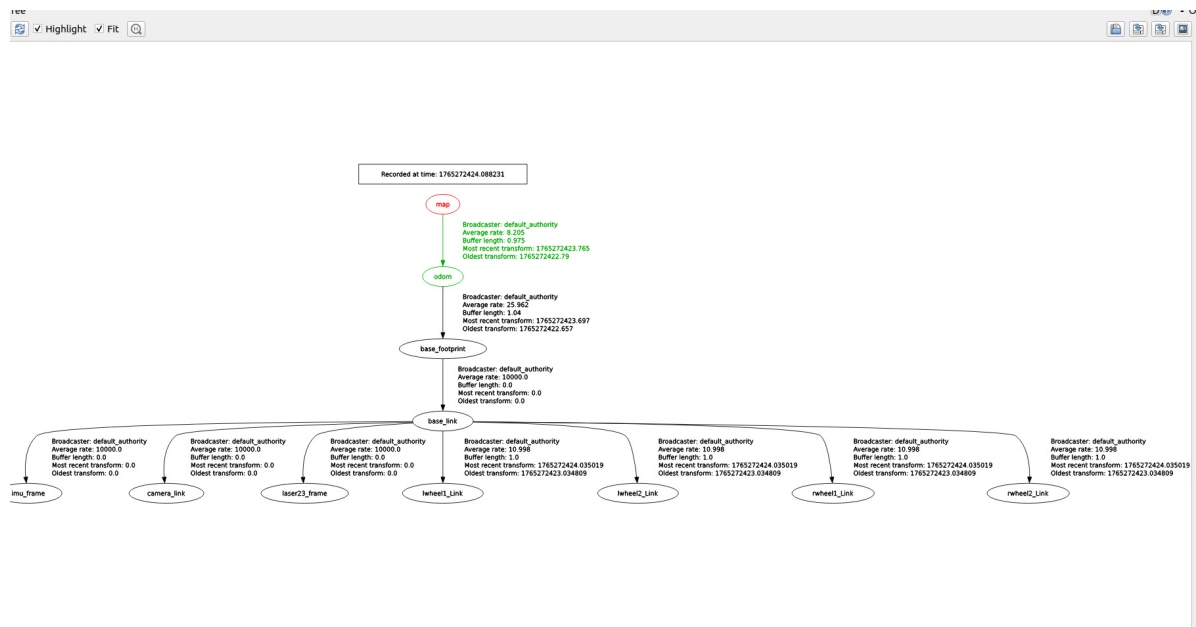
5. Source Code Analysis

5.1 View TF Tree

In the terminal, enter:

```
ros2 run rqt_tf_tree rqt_tf_tree
```

- Here, the TF transformation from the `map` coordinate frame to the `odom` coordinate frame is provided by `slam_toolbox`.



5.2 Launch File

- Launch file path:

```
~/yahboomM3_ws/user_ws/src/m3_bringup/launch/navigation/nav2.launch.py
```

- The key code snippet below starts the Navigation2 stack node and enables synchronous mapping mode, with the mapping function provided by the slam_toolbox algorithm.

```
# Navigate while building the map
nav_and_slam = GroupAction(
    condition=IfCondition(PythonExpression(["'", localization_mode, "' ==
'amcl' and '", slam, "' == 'True' "]])),
    actions=[
        # Navigation using AMCL positioning method
        PushRosNamespace(namespace=namespace),
        IncludeLaunchDescription(
            PythonLaunchDescriptionSource(nav2_launch_file),
            launch_arguments={
                "map": map_dir,
                "use_sim_time": use_sim_time,
                "params_file": params_file,
                'slam': 'True',
            }.items(),
        ),
        # Log Printing
        LogInfo(
            msg=[Fore.GREEN+'=====Robot-M3
Navigation_SLAM Start =====', Fore.RESET]
        )
    ]
)
```

- If you need to configure the slam_toolbox algorithm parameters for synchronous mapping and navigation, the path is as follows:

```
~/yahboomM3_ws/user_ws/src/m3_bringup/config/navigation_config/M3_nav2_params.yaml
```

```
#####  
# SLAM parameter configuration for simultaneous mapping and navigation  
#####  
slam_toolbox:  
  ros__parameters:  
  
    # Plugin params  
    solver_plugin: solver_plugins::CeresSolver  
    ceres_linear_solver: SPARSE_NORMAL_CHOLESKY  
    ceres_preconditioner: SCHUR_JACOBI  
    ceres_trust_strategy: LEVENBERG_MARQUARDT  
    ceres_dogleg_type: TRADITIONAL_DOGLEG  
    ceres_loss_function: None  
  
    # ROS Parameters  
    odom_frame: odom  
    map_frame: map  
    base_frame: base_footprint  
    scan_topic: /scan  
    use_map_saver: true  
    mode: mapping #localization  
  
    debug_logging: false  
    throttle_scans: 1  
    transform_publish_period: 0.02 #if 0 never publishes odometry  
    map_update_interval: 5.0  
    resolution: 0.05  
    min_laser_range: 0.05 #for rastering images  
    max_laser_range: 12.0 #for rastering images  
    minimum_time_interval: 0.5  
    transform_timeout: 0.2  
    tf_buffer_duration: 30.  
    stack_size_to_use: 40000000 #// program needs a larger stack size to  
    serialize large maps  
    enable_interactive_mode: true  
  
    # General Parameters  
    use_scan_matching: true  
    use_scan_barycenter: true  
    minimum_travel_distance: 0.5  
    minimum_travel_heading: 0.5  
    scan_buffer_size: 10  
    scan_buffer_maximum_scan_distance: 10.0  
    link_match_minimum_response_fine: 0.1  
    link_scan_maximum_distance: 1.5  
    loop_search_maximum_distance: 3.0
```

```
do_loop_closing: true
loop_match_minimum_chain_size: 10
loop_match_maximum_variance_coarse: 3.0
loop_match_minimum_response_coarse: 0.35
loop_match_minimum_response_fine: 0.45

# Correlation Parameters - Correlation Parameters
correlation_search_space_dimension: 0.5
correlation_search_space_resolution: 0.01
correlation_search_space_smear_deviation: 0.1

# Correlation Parameters - Loop Closure Parameters
loop_search_space_dimension: 8.0
loop_search_space_resolution: 0.05
loop_search_space_smear_deviation: 0.03

# Scan Matcher Parameters
distance_variance_penalty: 0.5
angle_variance_penalty: 1.0

fine_search_angle_offset: 0.00349
coarse_search_angle_offset: 0.349
coarse_angle_resolution: 0.0349
minimum_angle_penalty: 0.9
minimum_distance_penalty: 0.5
use_response_expansion: true
min_pass_through: 2
occupancy_threshold: 0.1
```